

尚硅谷嵌入式技术之 C 语言（项目）

（作者：尚硅谷研究院）

版本：V1.6.0

第 1 章 项目——客户信息管理系统

实现一个客户信息管理系统，功能包括添加客户、修改客户、删除客户、显示客户列表。

1.1 需求说明

1.1.1 主菜单

进入系统，展示主菜单，输入各功能对应的数字编号选择要进行的操作，如下图：

-----客户信息管理软件-----
1 添加客户
2 修改客户
3 删除客户
4 客户列表
5 退出
请选择（1-5）：

1.1.2 添加客户

输入 1，进入“添加客户”界面，需要填写姓名、性别、年龄、电话、邮箱这些信息，如下图：

```

-----客户信息管理软件-----
1 添加客户
2 修改客户
3 删除客户
4 客户列表
5 退出
请选择（1-5）： 1

-----添加客户-----
姓名： Jack

性别(m或f)： m

年龄： 56

电话： 110

邮箱： jack@qq.com

添加成功！
    
```

1.1.3 显示客户列表

输入 4，进入“客户列表”，显示客户列表信息，如下图：

```

-----客户信息管理软件-----
1 添加客户
2 修改客户
3 删除客户
4 客户列表
5 退出
请选择（1-5）： 4

-----客户列表-----
编号   姓名   性别   年龄   电话   邮箱
1      Jack   m      56     110    jack@qq.com
2      Tony   f      42     119    tony@163.com
    
```

1.1.4 删除客户

输入 3，进入“删除客户”，输入要删除客户的编号，再确认是否删除，如下图：

```

-----客户信息管理软件-----
1 添加客户
2 修改客户
3 删除客户
4 客户列表
5 退出
请选择（1-5）： 3

-----删除客户-----
输出要删除客户的编号（-1退出）： 1

再次确认删除(Y/N)： Y

删除成功！
    
```

1.1.5 修改客户

输入 2，进入“修改客户”，首先要输入要修改客户的编号，然后填写信息，如下图：

```

-----客户信息管理软件-----
1 添加客户
2 修改客户
3 删除客户
4 客户列表
5 退出
请选择（1-5）： 2

-----修改客户-----
请输入要修改客户的编号(-1退出)： 2

姓名（Tony）： 注意：如果不需要修改直接输入回车键！

年龄（42）： 43

性别（f）： m

电话（119）：

邮箱（tony@163.com）：

修改成功！
    
```

1.1.6 退出

输出 5，经过确认后退出系统，如下图：

-----客户信息管理软件-----

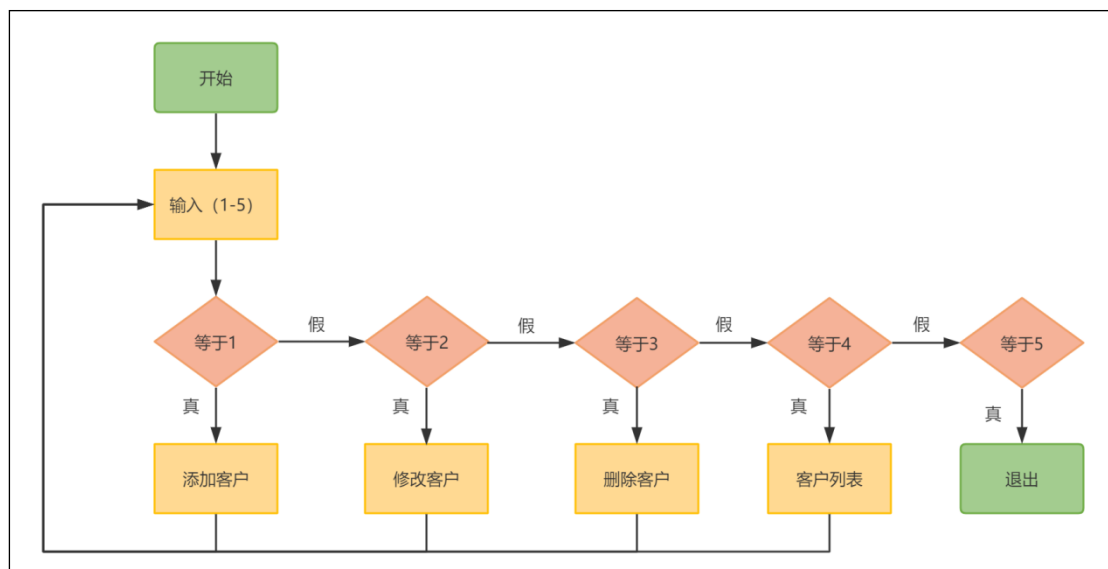
- 1 添加客户
 - 2 修改客户
 - 3 删除客户
 - 4 客户列表
 - 5 退出
- 请选择（1-5）： 5

是否确认退出（Y或N）： Y

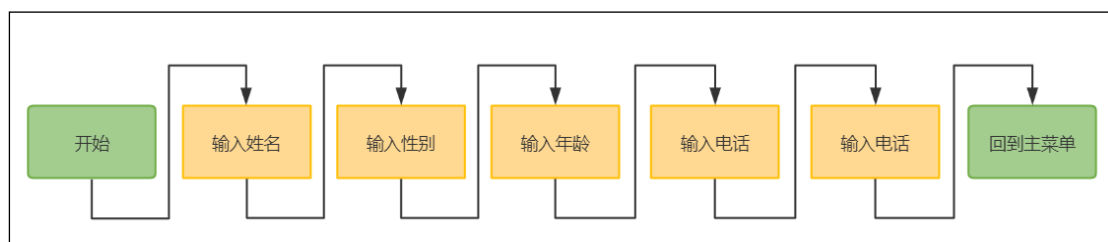
退出系统！

1.2 流程分析

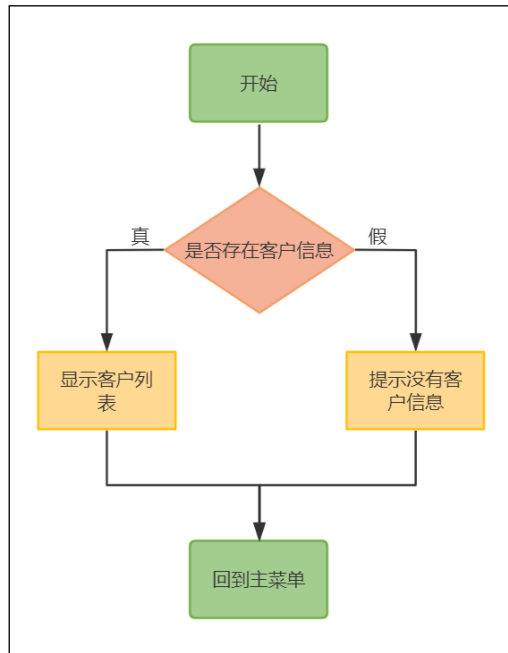
1.2.1 总流程图



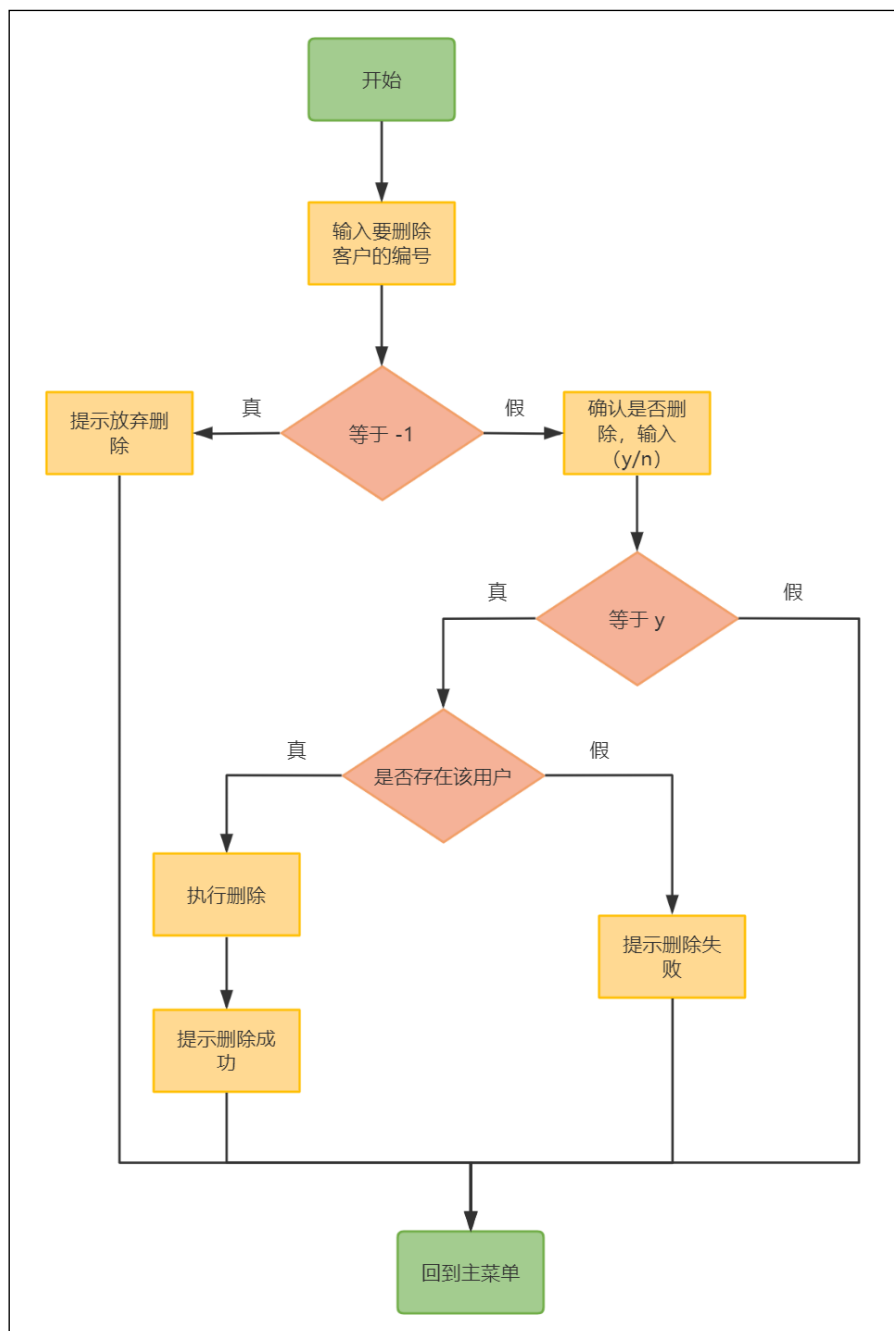
1.2.2 添加客户流程图



1.2.3 显示客户列表流程图

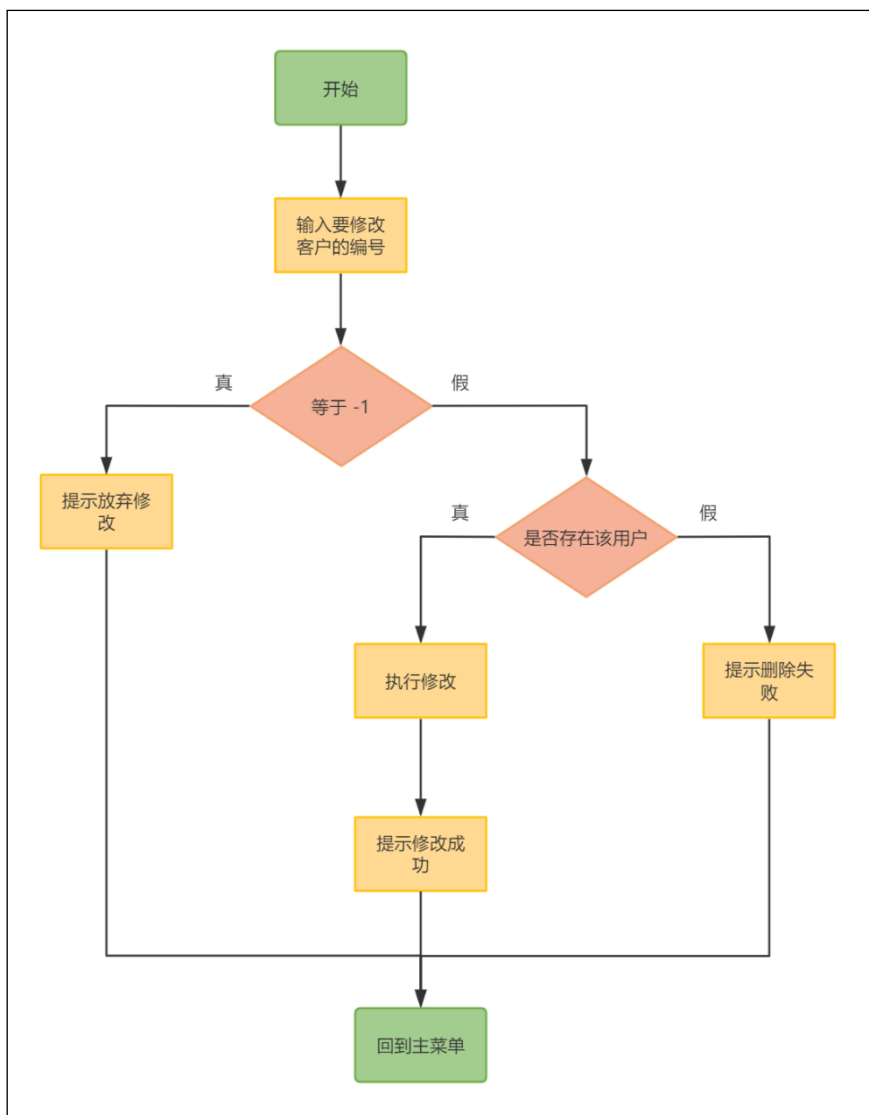


1.2.4 删除客户流程图

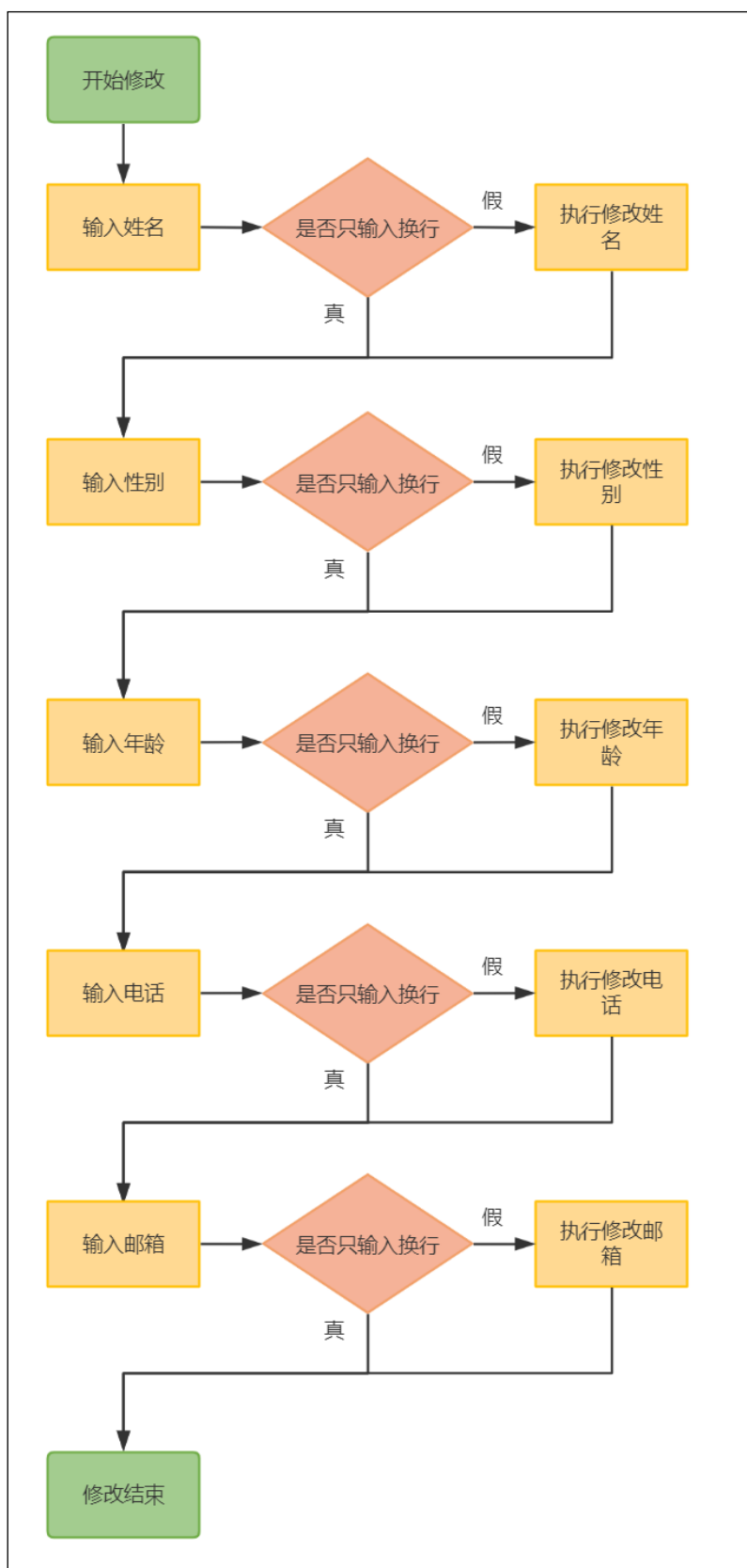


1.2.5 修改客户流程图

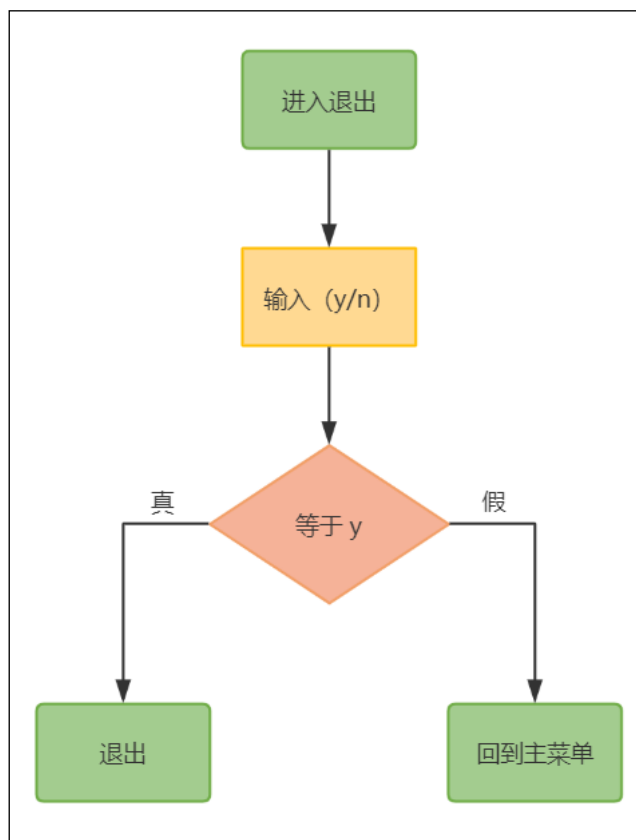
(1) 修改客户总体流程图:



(2) 具体执行修改部分的流程图：



1.2.6 退出流程图



1.3 代码实现

1.3.1 主菜单和退出

```
#include <stdio.h>

// 实现主菜单
void menu()
{
    // 定义循环变量
    int loop = 1;

    // 启动循环
    while (loop)
    {
        printf("\n-----客户信息管理软件-----");
        printf("\n          1 添加客户");
        printf("\n          2 修改客户");
        printf("\n          3 删除客户");
        printf("\n          4 客户列表");
        printf("\n          5 退出");
    }
}
```

```
printf("\n                请选择（1-5）：");

// 获取用户选择
int choose;
scanf("%d", &choose);

// 根据用户选择进行不同操作
switch (choose)
{
case 1:
    printf("\n 添加客户！");
    break;
case 2:
    printf("\n 修改客户！");
    break;
case 3:
    printf("\n 删除客户！");
    break;
case 4:
    printf("\n 客户列表！");
    break;
case 5:
    printf("\n 是否确认退出（Y 或 N）：");
    char ch;
    getchar(); // 过滤换行
    scanf("%c", &ch);
    if (ch == 'Y' || ch == 'y')
    {
        printf("\n 退出系统！");
        loop = 0;
    }
    break;
default:
    printf("\n 输入有误！");
}
}

int main()
{
    // 主菜单
    menu();
    return 0;
}
```

1.3.2 添加客户

```
// 定义表示客户的结构体
typedef struct
{
    int id;           // 客户编号
    char name[20];    // 客户姓名
    char gender;      // 客户性别, 值是 'f' 或 'm'
    int age;          // 客户年龄
    char phone[20];   // 客户电话
    char email[50];   // 客户邮箱
} Customer;

// 声明 Customer 类型的数组, 用于存储客户信息, 最多存储 100 个客户
Customer customers[100];
// 定义变量记录已经存储的用户个数
int customer_count = 0;

// 添加客户
void add()
{
    // 如果超出最大可存储个数, 不再添加
    if (customer_count >= 100)
    {
        printf("\n 客户已满! ");
        return;
    }

    // 设置编号
    customers[customer_count].id = customer_count + 1;

    printf("\n-----添加客户-----");
    printf("\n 姓名: ");
    scanf("%s", customers[customer_count].name);

    printf("\n 性别(m 或 f): ");
    getchar(); // 过滤换行
    scanf("%c", &(customers[customer_count].gender));

    printf("\n 年龄: ");
    scanf("%d", &(customers[customer_count].age));

    printf("\n 电话: ");
```

```
scanf("%s", customers[customer_count].phone);

printf("\n 邮箱: ");
scanf("%s", customers[customer_count].email);

printf("\n 添加成功! ");

// 已存储个数自增
customer_count++;
}
```

1.3.3 显示客户列表

```
// 客户列表
void list()
{
    printf("\n-----客户列表-----");
    // 如果没有客户信息，退出函数
    if (customer_count < 1)
    {
        printf("\n 还没有客户，快去添加吧! ");
        return;
    }
    // 循环输出客户信息
    printf("\n 编号\t姓名\t性别\t年龄\t电话\t邮箱");
    for (int i = 0; i < customer_count; i++)
    {
        printf("\n%d\t%s\t%c\t%d\t%s\t%s", customers[i].id,
customers[i].name, customers[i].gender, customers[i].age,
customers[i].phone, customers[i].email);
    }
}
```

1.3.4 删除客户

```
// 执行删除
// 参数 要删除客户的编号
// 返回值 1 表示删除成功 0 表示删除失败
int delete(int id)
{
    // 判断编号有效
    if (id < 1 || id > customer_count)
    {
        return 0;
    }
}
```

```
// 根据编号计算要删除元素(客户)的下标
int index = id - 1;

// 从要删除的元素开始,后面的元素依次向前移动
// 循环 从 index 开始,遍历到倒数第二个元素
for (int i = index; i < customer_count - 1; i++)
{
    // 将后面的元素赋值给当前元素
    customers[i] = customers[i + 1];
    // 修改编号,自减
    customers[i].id--;
}

// 当前客户数 自减
customer_count--;
return 1;
}

// 删除界面
void delete_view()
{
    printf("\n-----删除客户-----");
    printf("\n 输出要删除客户的编号 (-1 退出) : ");

    int id;
    scanf("%d", &id);

    // 如果输入-1 退出
    if (id == -1)
    {
        printf("\n 放弃删除! ");
        return;
    }

    printf("\n 再次确认删除(Y/N): ");
    char ch;
    getchar(); // 过滤换行
    scanf("%c", &ch);

    // 判断是否确认删除
    if (ch == 'Y' || ch == 'y')
    {
        if (delete (id))
        {

```

```
        printf("\n 删除成功! ");
    }
    else
    {
        printf("\n 删除失败! ");
    }
}
}
```

1.3.5 修改客户

```
// 执行修改
// 参数: 要修改客户的编号
// 返回值: 1 表示修改成功 0 表示修改失败
int modify(int id)
{
    // 判断 编号在合理范围内
    if (id < 1 || id > customer_count)
    {
        return 0;
    }

    // 计算要修改元素(客户)的下标
    int index = id - 1;

    // 定义字符数组 获取输入内容
    char buff[50];

    // 修改姓名
    printf("\n 姓名 (%s): ", customers[index].name);
    getchar(); // 获取到上次输入的换行
    fgets(buff, sizeof buff, stdin);
    // 用户输入的不是换行, 才修改
    if (*buff != '\n')
    {
        sscanf(buff, "%s", customers[index].name);
    }

    // 修改年龄
    printf("\n 年龄 (%d): ", customers[index].age);
    fgets(buff, sizeof buff, stdin);
    // 用户输入的不是换行, 才修改
    if (*buff != '\n')
    {
        sscanf(buff, "%d", &customers[index].age);
    }
}
```

```
}

// 修改性别
printf("\n 性别 (%c) : ", customers[index].gender);
fgets(buff, sizeof buff, stdin);
// 用户输入的不是换行, 才修改
if (*buff != '\n')
{
    sscanf(buff, "%c", &customers[index].gender);
}

// 修改电话
printf("\n 电话 (%s) : ", customers[index].phone);
fgets(buff, sizeof buff, stdin);
// 用户输入的不是换行, 才修改
if (*buff != '\n')
{
    sscanf(buff, "%s", customers[index].phone);
}

// 修改邮箱
printf("\n 邮箱 (%s) : ", customers[index].email);
fgets(buff, sizeof buff, stdin);
// 用户输入的不是换行, 才修改
if (*buff != '\n')
{
    sscanf(buff, "%s", customers[index].email);
}

return 1;
}

// 修改界面
void modify_view()
{
    printf("\n-----修改客户-----");
    printf("\n 请输入要修改客户的编号(-1退出): ");

    int id;
    scanf("%d", &id);

    if (id == -1)
    {
        printf("\n 放弃修改! ");
    }
}
```

```
        return;
    }

    // 执行修改
    if (modify(id))
    {
        printf("\n 修改成功! ");
    }
    else
    {
        printf("\n 修改失败! ");
    }
}
```

1.3.6 全部代码

```
#include <stdio.h>

// 定义表示客户的结构体
typedef struct
{
    int id;           // 客户编号
    char name[20];    // 客户姓名
    char gender;      // 客户性别, 值是 'f' 或 'm'
    int age;          // 客户年龄
    char phone[20];   // 客户电话
    char email[50];   // 客户邮箱
} Customer;

// 声明结构体数组 存储客户
Customer customers[100];
// 定义变量记录已经存储的用户个数
int customer_count = 0;

// 执行修改
// 参数: 要修改客户的编号
// 返回值: 1 表示修改成功 0 表示修改失败
int modify(int id)
{
    // 判断 编号在合理范围内
    if (id < 1 || id > customer_count)
    {
        return 0;
    }
}
```



```
// 计算要修改元素（客户）的下标
int index = id - 1;

// 定义字符数组 获取输入内容
char buff[50];

// 修改姓名
printf("\n 姓名 (%s) : ", customers[index].name);
getchar(); // 获取到上次输入的换行
fgets(buff, sizeof buff, stdin);
// 用户输入的不是换行，才修改
if (*buff != '\n')
{
    sscanf(buff, "%s", customers[index].name);
}

printf("\n 年龄 (%d) : ", customers[index].age);
fgets(buff, sizeof buff, stdin);
// 用户输入的不是换行，才修改
if (*buff != '\n')
{
    sscanf(buff, "%d", &customers[index].age);
}

printf("\n 性别 (%c) : ", customers[index].gender);
fgets(buff, sizeof buff, stdin);
// 用户输入的不是换行，才修改
if (*buff != '\n')
{
    sscanf(buff, "%c", &customers[index].gender);
}

printf("\n 电话 (%s) : ", customers[index].phone);
fgets(buff, sizeof buff, stdin);
// 用户输入的不是换行，才修改
if (*buff != '\n')
{
    sscanf(buff, "%s", customers[index].phone);
}

printf("\n 邮箱 (%s) : ", customers[index].email);
fgets(buff, sizeof buff, stdin);
// 用户输入的不是换行，才修改
```

```
    if (*buff != '\n')
    {
        sscanf(buff, "%s", customers[index].email);
    }

    return 1;
}

// 修改界面
void modify_view()
{
    printf("\n-----修改客户-----");
    printf("\n 请输入要修改客户的编号(-1 退出): ");

    int id;
    scanf("%d", &id);

    if (id == -1)
    {
        printf("\n 放弃修改! ");
        return;
    }

    // 执行修改
    if (modify(id))
    {
        printf("\n 修改成功! ");
    }
    else
    {
        printf("\n 修改失败! ");
    }
}

// 执行删除
// 参数 要删除客户的编号
// 返回值 1 表示删除成功 0 表示删除失败
int delete(int id)
{
    // 判断编号有效
    if (id < 1 || id > customer_count)
    {
        return 0;
    }
}
```

```
// 根据编号计算要删除元素(客户)的下标
int index = id - 1;

// 从要删除的元素开始,后面的元素依次向前移动
// 循环 从 index 开始,遍历到倒数第二个元素
for (int i = index; i < customer_count - 1; i++)
{
    // 将后面的元素赋值给当前元素
    customers[i] = customers[i + 1];
    // 修改编号,自减
    customers[i].id--;
}

// 当前客户数 自减
customer_count--;
return 1;
}

// 删除界面
void delete_view()
{
    printf("\n-----删除客户-----");
    printf("\n输出要删除客户的编号 (-1 退出): ");

    int id;
    scanf("%d", &id);

    // 如果输入-1 退出
    if (id == -1)
    {
        printf("\n放弃删除! ");
        return;
    }

    printf("\n再次确认删除(Y/N): ");
    char ch;
    getchar(); // 获取上次输入的换行
    scanf("%c", &ch);

    // 判断是否确认删除
    if (ch == 'Y' || ch == 'y')
    {
        if (delete (id))
```

```
        {
            printf("\n 删除成功! ");
        }
        else
        {
            printf("\n 删除失败! ");
        }
    }
}

// 客户列表
void list()
{
    printf("\n-----客户列表-----");
    // 如果没有客户信息，退出函数
    if (customer_count < 1)
    {
        printf("\n 还没有客户，快去添加吧! ");
        return;
    }
    // 循环输出客户信息
    printf("\n 编号\t姓名\t性别\t年龄\t电话\t邮箱");
    for (int i = 0; i < customer_count; i++)
    {
        printf("\n%d\t%s\t%c\t%d\t%s\t%s", customers[i].id,
customers[i].name, customers[i].gender, customers[i].age,
customers[i].phone, customers[i].email);
    }
}

// 添加客户
void add()
{
    // 如果已满，不再添加
    if (customer_count >= 100)
    {
        printf("\n 客户已满! ");
        return;
    }

    // 设置编号
    customers[customer_count].id = customer_count + 1;

    printf("\n-----添加客户-----");
```

```
printf("\n 姓名: ");
scanf("%s", customers[customer_count].name);
printf("\n 性别(m 或 f): ");
getchar();
scanf("%c", &(customers[customer_count].gender));
printf("\n 年龄: ");
scanf("%d", &(customers[customer_count].age));
printf("\n 电话: ");
scanf("%s", customers[customer_count].phone);
printf("\n 邮箱: ");
scanf("%s", customers[customer_count].email);

printf("\n 添加成功! ");

// 已存储个数自增
customer_count++;
}

// 实现主菜单
void menu()
{
    // 定义循环变量
    int loop = 1;

    // 启动循环
    while (loop)
    {
        printf("\n-----客户信息管理软件-----");
        printf("\n          1 添加客户");
        printf("\n          2 修改客户");
        printf("\n          3 删除客户");
        printf("\n          4 客户列表");
        printf("\n          5 退出");
        printf("\n          请选择 (1-5): ");

        // 获取用户选择
        int choose;
        scanf("%d", &choose);

        // 根据用户选择进行不同操作
        switch (choose)
        {
            case 1:
                add();
```

```
        break;
    case 2:
        modify_view();
        break;
    case 3:
        delete_view();
        break;
    case 4:
        list();
        break;
    case 5:
        printf("\n 是否确认退出 (Y 或 N) : ");
        char ch;
        getchar(); // 过滤换行
        scanf("%c", &ch);
        if (ch == 'Y' || ch == 'y')
        {
            printf("\n 退出系统! ");
            loop = 0;
        }
        break;
    default:
        printf("\n 输入有误! ");
    }
}

int main()
{
    // 主菜单
    menu();
    return 0;
}
```